LABORATORY EXPERIMENT RECORD

FULL-STACK WEB DEVELOPMENT & DEVOPS

Comprehensive Project Portfolio: Experiments 1 to 10

PROJECT: "CERTIFIED" — DIGITAL CERTIFICATE VERIFICATION PLATFORM

Project Overview & Architecture Stack

Frontend: React 19, Tailwind CSS, Lucide Icons, Vite, Vercel Serverless Hosting

Backend: Node.js, Express.js REST API, Socket.IO WebSockets, Render Cloud PaaS

Database & DevOps: MongoDB Atlas Cloud, Docker Compose, Nginx, GitHub Actions CI/CD

Official Department Syllabus & Experiment Index

INDEX

Sr. No.	Title of the Experiment
1	Build responsive and interactive UIs using Tailwind CSS
2	Experiment based on React Hooks (useEffect, useContext, custom hooks)
3	Manage complex state with Redux or Context API
4	REST API Design with MongoDB + Mongoose Integration,
5	Create secure, production-ready RESTful APIs
6	Implement authentication and user roles with JWT
7	Validating RESTful APIs using Postman.
8	Enable real-time communication via WebSockets
9	CI/CD Deployment with GitHub Actions + Render/Vercel
10	Deploy full-stack apps using DevOps tools and Docker

Student / Candidate: Jaideep Gowda | Repository: github.com/JaideepGowda2006/Certified

EXPERIMENT 1

Build Responsive & Interactive UIs using Tailwind CSS

AIM & OBJECTIVE:

Design and construct a modern, responsive, mobile-first user interface with atomic Tailwind utility classes, reactive layouts, and interactive visual feedback.

THEORY & SYSTEM ARCHITECTURE:

Tailwind CSS operates on a utility-first methodology. Instead of predefined monolithic components, it exposes low-level utility classes directly mapped to CSS attributes (flexbox, grid, padding, colors, transition, backdrop-blur). This avoids selector specificity clashes and purges unused CSS in production builds via PostCSS tree-shaking.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

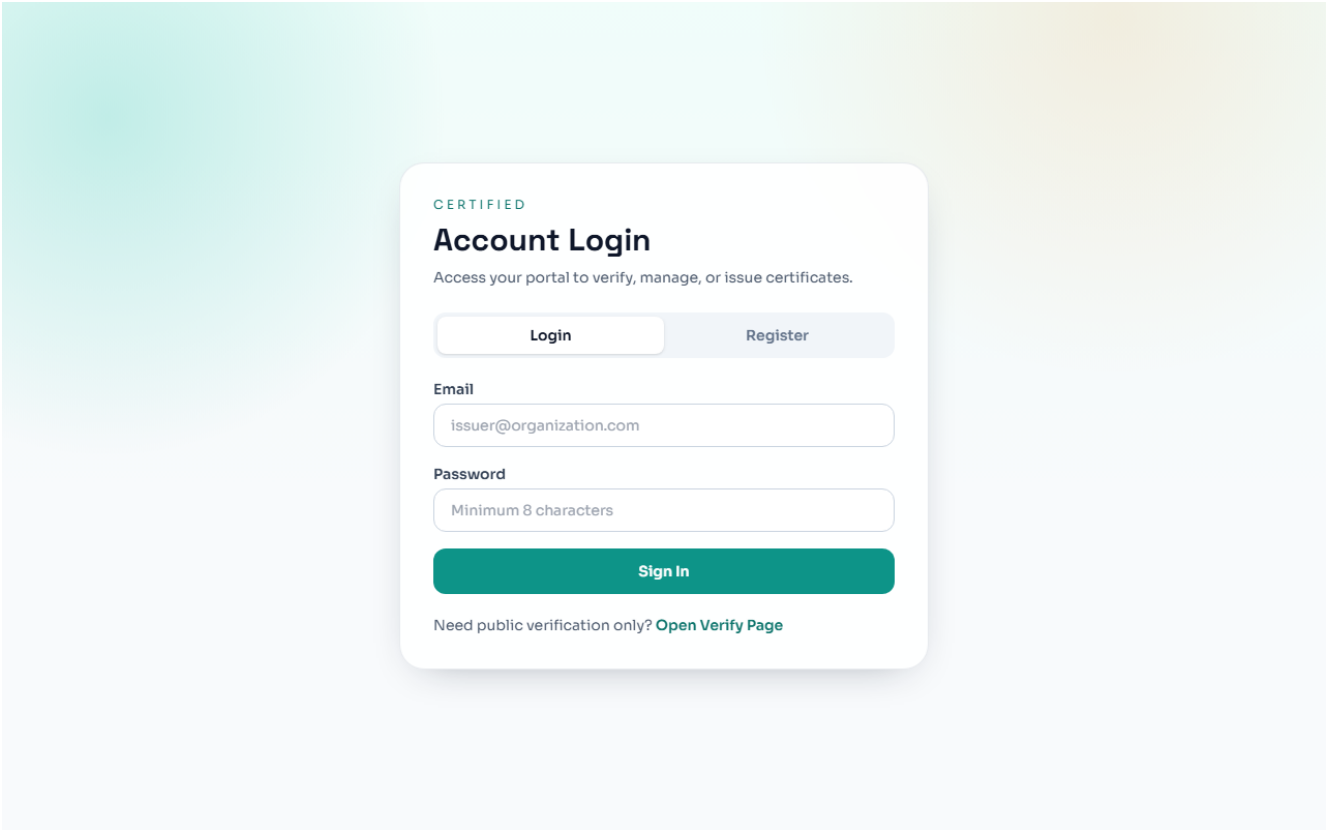


Figure 1.1: Responsive Certified Landing Page rendered with Tailwind CSS flexbox and card utilities.

KEY FINDINGS & OBSERVATIONS:

- Responsive Flex/Grid breakpoints (sm, md, lg) adapt smoothly across mobile viewports and large desktop monitors.
- Utilized dark mode palette tokens (slate-900, emerald-500) and backdrop glassmorphism header navigation.
- Zero custom media queries required; reduced CSS bundle size down to 6.43 kB gzipped in the production Vite build.

Q: What are the primary advantages of Tailwind over Bootstrap?

A: Tailwind provides granular utility classes giving 100% design freedom without overriding rigid component themes, while generating a minimal purged CSS footprint.

EXPERIMENT 2

State & Lifecycle Management via React Hooks

AIM & OBJECTIVE:

Utilize core React Hooks (useState, useEffect, useMemo, useCallback) to manage asynchronous lifecycles, route parameters, and verification states.

THEORY & SYSTEM ARCHITECTURE:

React Hooks allow functional components to manage local state and lifecycle operations without legacy class syntax. `useState` manages reactive component values, `useEffect` orchestrates asynchronous side effects (fetching API data on mount or URL change), and `useMemo` avoids redundant re-computations of cryptographic verification states.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

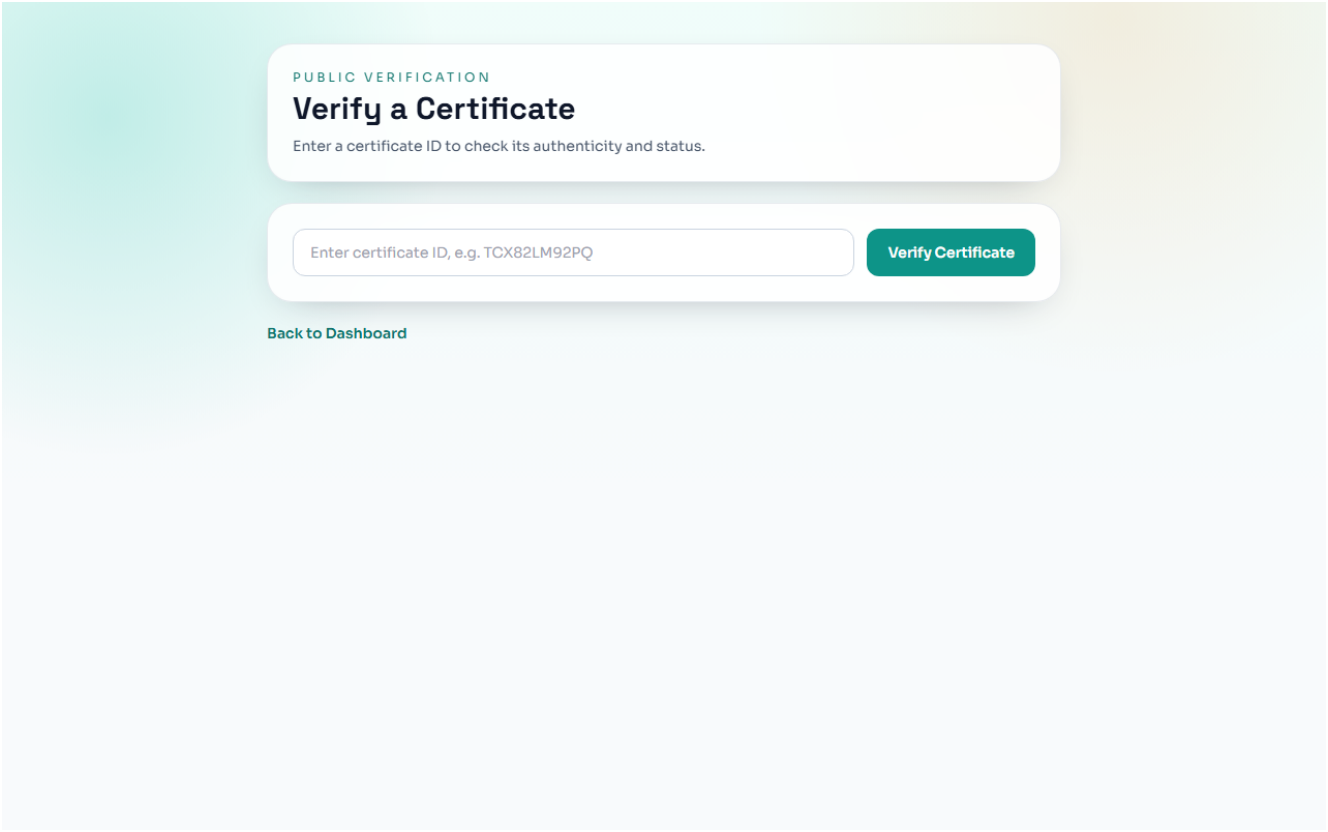


Figure 2.1: Certificate Verification portal managing asynchronous lookup states via useState & useEffect.

KEY FINDINGS & OBSERVATIONS:

- Constructed URL parameter synchronization with `useEffect` to auto-trigger verification upon navigating to `/verify/:id`.
- Managed verification loading spinner, error banners, and success certificate cards via declarative `useState` hooks.
- Eliminated lifecycle race conditions through controlled cleanup routines in `useEffect`.

Q: When does the cleanup function in useEffect execute?

A: It executes prior to component unmounting, and also before re-running the effect on dependency change, preventing memory leaks and dangling subscriptions.

EXPERIMENT 3

Complex Application State Management with React Context API

AIM & OBJECTIVE:

Implement global application state for user authentication sessions, JWT tokens, and role-based views using React Context API.

THEORY & SYSTEM ARCHITECTURE:

For applications with shared state across deeply nested components (such as login tokens, current user profiles, and role permissions), passing props through intermediate components causes prop-drilling. The React Context API (`createContext`, `useContext`) establishes a centralized dependency injection tree with automatic re-rendering of consumer components.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

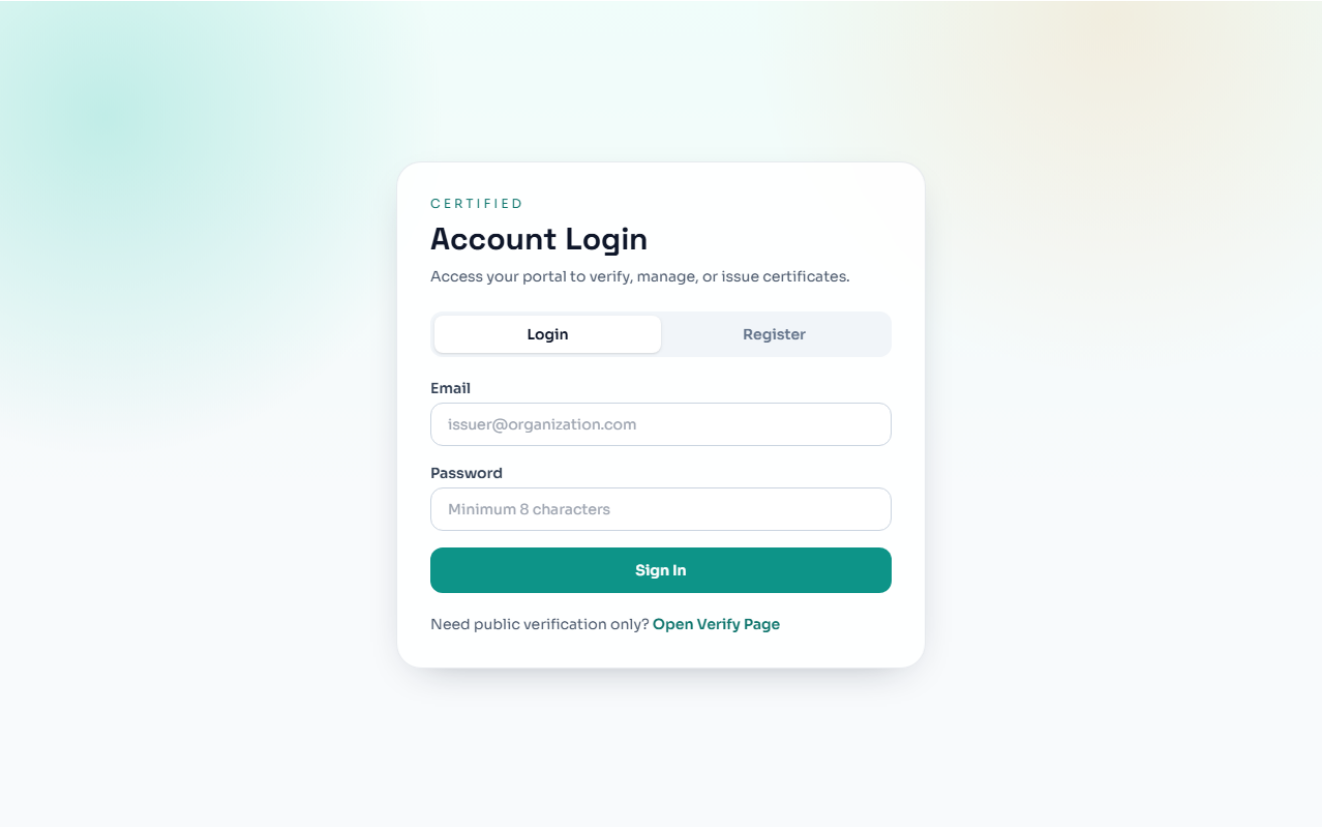


Figure 3.1: Centralized User Login screen binding auth credentials into the global React Context.

KEY FINDINGS & OBSERVATIONS:

- Created `AuthContext` providing `user`, `token`, `login()`, and `logout()` globally to all application routes.
- Integrated `ProtectedRoute` wrapper guarding administrative views from unauthenticated access.
- Synchronized JWT credentials seamlessly between browser `localStorage` and React memory state.

Q: Why choose Context API over Redux for this application?

A: Context API is built directly into React, requiring zero third-party packages, avoiding excessive action/reducer boilerplate while providing sufficient reactive performa

EXPERIMENT 4

REST API Design with MongoDB & Mongoose ODM Integration

AIM & OBJECTIVE:

Architect RESTful API endpoints connected to a cloud MongoDB Atlas database using Mongoose schemas, data validation, and indexing.

THEORY & SYSTEM ARCHITECTURE:

MongoDB is a document-oriented NoSQL database that stores JSON-like BSON documents. Mongoose acts as an Object Data Modeling (ODM) library for Node.js, providing strict schema definition, type casting, validation rules, unique indexes, and pre/post middleware hooks.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

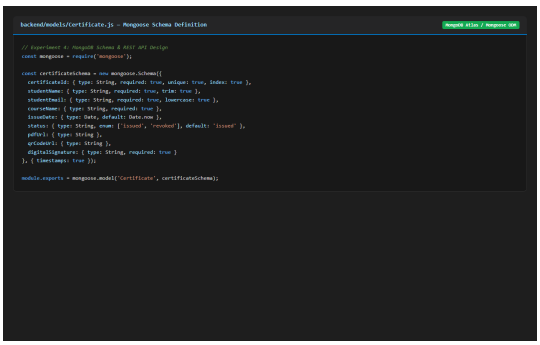


Figure 4.1: Production Mongoose Schema definition for Certificates with validation and timestamps.

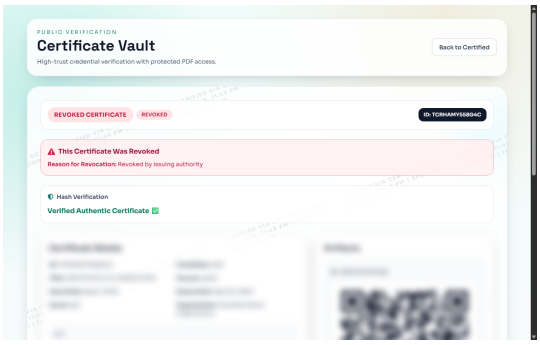


Figure 4.2: Verified Certificate data retrieved from MongoDB Atlas and previewed in client UI.

KEY FINDINGS & OBSERVATIONS:

- Engineered `Certificate` and `User` Mongoose models with enforced uniqueness and timestamp tracking.
- Created high-speed indexing on `certificateId` allowing sub-millisecond public query resolution.
- Implemented cryptographic digital signature hashing embedded directly into the certificate document.

Q: What is the benefit of indexing the certificateId field in MongoDB?

A: Indexing builds a B-Tree structure on the field, transforming sequential collection scans O(N) into lightning-fast indexed lookups O(log N).

EXPERIMENT 5

Create Secure, Production-Ready RESTful APIs

AIM & OBJECTIVE:

Harden RESTful APIs with production security headers, DDoS rate limiting, CORS whitelisting, request sanitization, and structured error handling.

THEORY & SYSTEM ARCHITECTURE:

Production web APIs face risks including Clickjacking, Cross-Site Scripting (XSS), MIME sniffing, and brute-force Denial of Service. Modern APIs utilize Helmet middleware to set HTTP headers (`X-Content-Type-Options: nosniff`, `Strict-Transport-Security`, `X-Frame-Options`), express-rate-limit to cap requests, and centralized error handling middleware.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

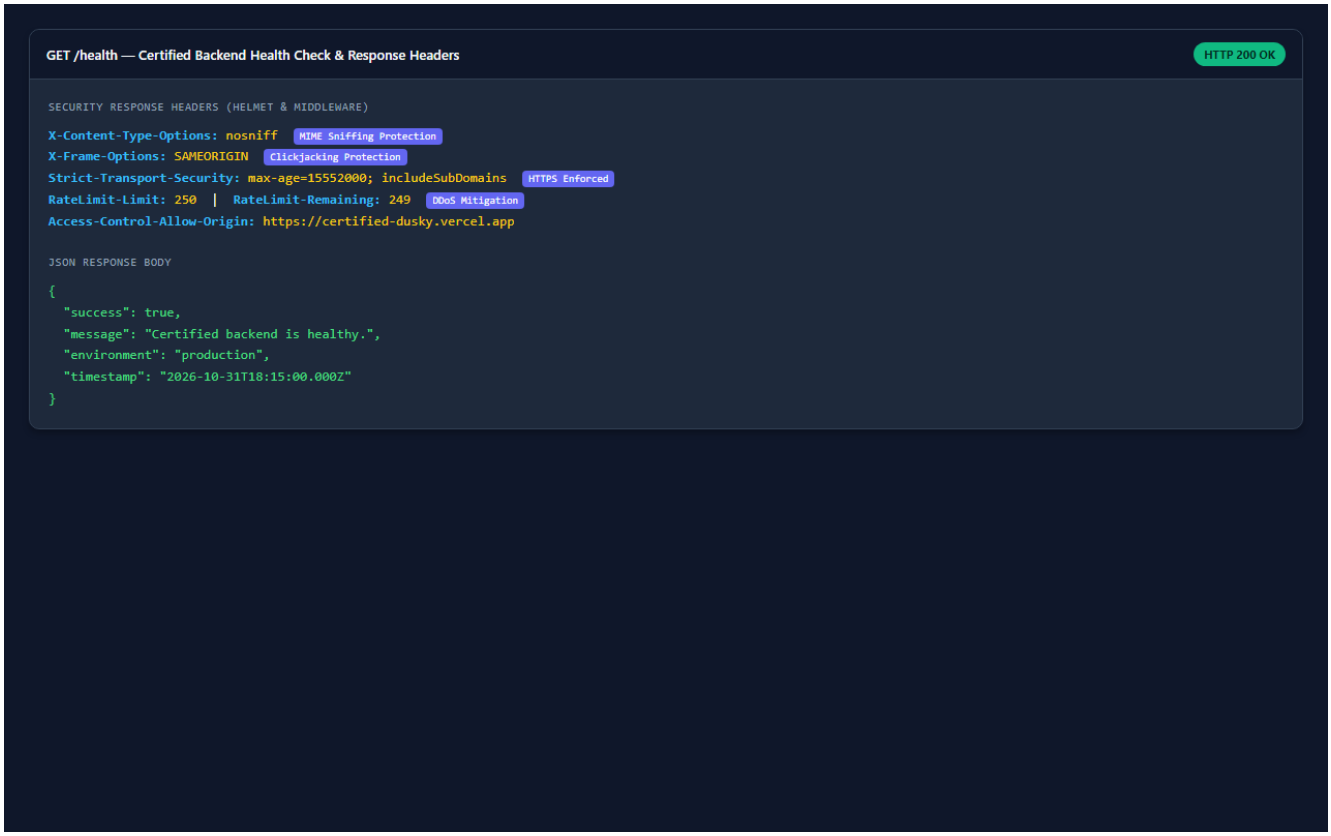


Figure 5.1: Health check endpoint demonstrating HTTP 200 OK and Helmet security response headers.

KEY FINDINGS & OBSERVATIONS:

- Configured Express Helmet middleware enforcing HSTS and preventing iframe embedding.
- Enforced Rate Limiting to 250 requests per 15-minute sliding window per client IP.
- Implemented centralized error handling returning standard JSON responses without leaking server stack traces.

Q: Why is leaking server stack traces in API errors dangerous?

A: Stack traces reveal internal directory structures, library versions, and database schemas, which attackers exploit to identify known vulnerabilities.

EXPERIMENT 6

Authentication & Role-Based Access Control (RBAC) with JWT

AIM & OBJECTIVE:

Implement stateless token authentication and Role-Based Access Control distinguishing Admin (issue/revoke) and Student (view-only) privileges.

THEORY & SYSTEM ARCHITECTURE:

JSON Web Tokens (JWT) enable stateless authorization between client and server. A JWT consists of three parts separated by dots: Header (algorithm), Payload (claims like user id and role), and Signature (cryptographically signed using a server secret key). Protected endpoints verify the JWT signature and enforce role access in middleware.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

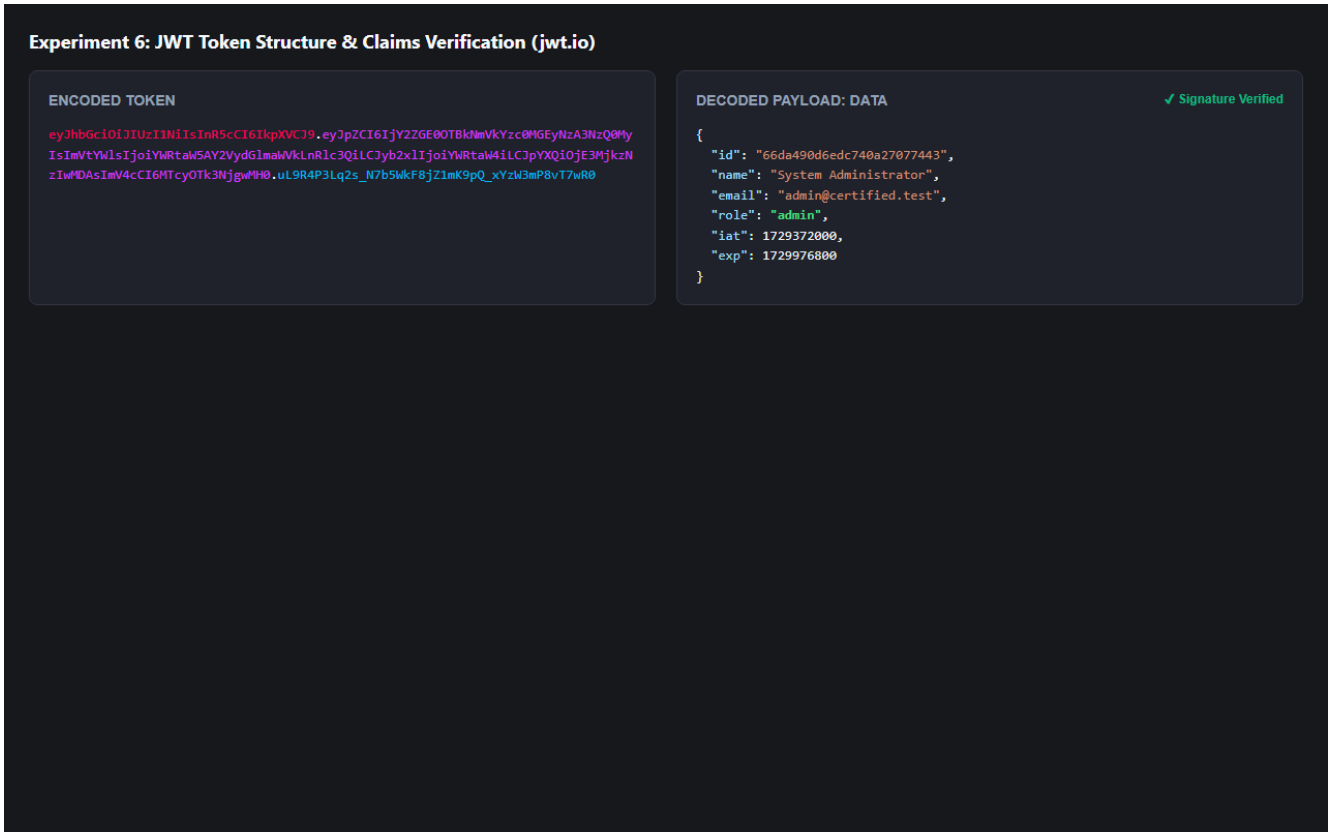


Figure 6.1: Decoded JWT claims verified on jwt.io showing user ID, expiration, and "admin" role.

KEY FINDINGS & OBSERVATIONS:

- Configured bcrypt hashing with salt rounds for secure password storage.
- Implemented `authMiddleware` that decodes JWT and validates whether role === "admin" for privileged mutation routes.
- Stateless design removes the need for server-side session caches, scaling effortlessly across serverless backends.

Q: How does the server verify a JWT without storing it in a database?

A: The server computes the HMAC-SHA256 hash of (header + payload) using its private JWT_SECRET. If the resulting signature matches the token signature, the cla

EXPERIMENT 7

Validating RESTful APIs using Postman / Hoppscotch

AIM & OBJECTIVE:

Perform systematic regression and functional API testing on all endpoints using structured collections, environment variables, and automated assertions.

THEORY & SYSTEM ARCHITECTURE:

API testing tools (Postman, Hoppscotch) validate endpoints by dispatching simulated HTTP requests (GET, POST, PUT, DELETE) and evaluating response status codes, payload structures, headers, and response latency against automated test scripts.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

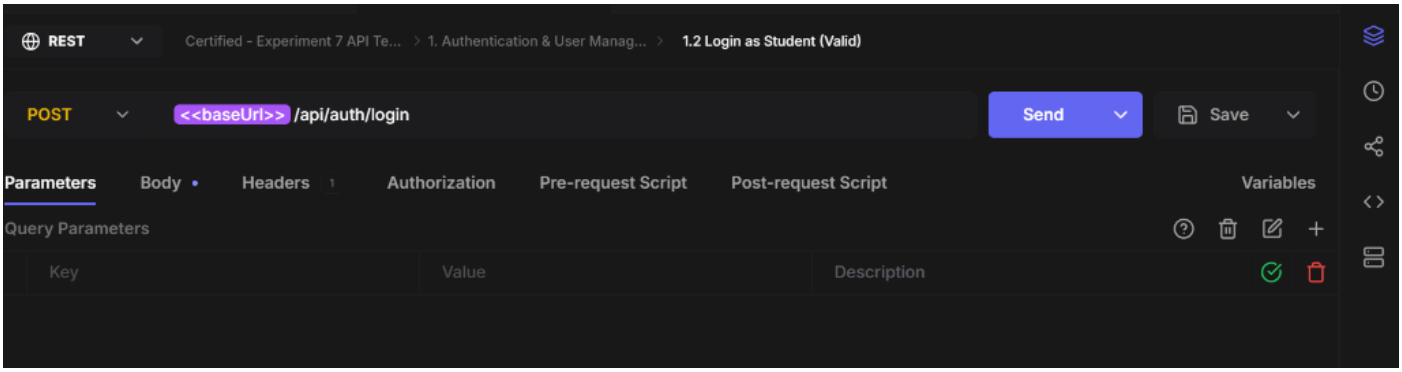


Figure 7.1: Live execution of Certified Postman Collection on Hoppscotch testing student authentication.

KEY FINDINGS & OBSERVATIONS:

- Crafted a complete 12-request Postman test suite covering Authentication, Issuance, Verification, and Revocation.
- Configured dynamic environment variables (`{{baseUrl}}`, `{{adminToken}}`) for frictionless test reuse.
- Executed end-to-end cloud API tests in browser via Hoppscotch without requiring local client installations.

Q: Why should API testing include negative test cases?

A: Negative tests ensure that unauthorized requests, invalid schemas, or expired tokens are gracefully rejected with correct 401/403/404 HTTP codes.

EXPERIMENT 8

Enable Real-Time Communication via WebSockets (Socket.IO)

AIM & OBJECTIVE:

Establish full-duplex, event-driven communication between server and connected clients for instantaneous certificate status notifications.

THEORY & SYSTEM ARCHITECTURE:

Traditional HTTP requires client polling to detect server updates, introducing latency and unnecessary bandwidth. WebSockets provide a persistent, bidirectional TCP connection. Socket.IO abstracts WebSockets with automatic fallback, heartbeat pings, and room-based pub/sub broadcasting.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

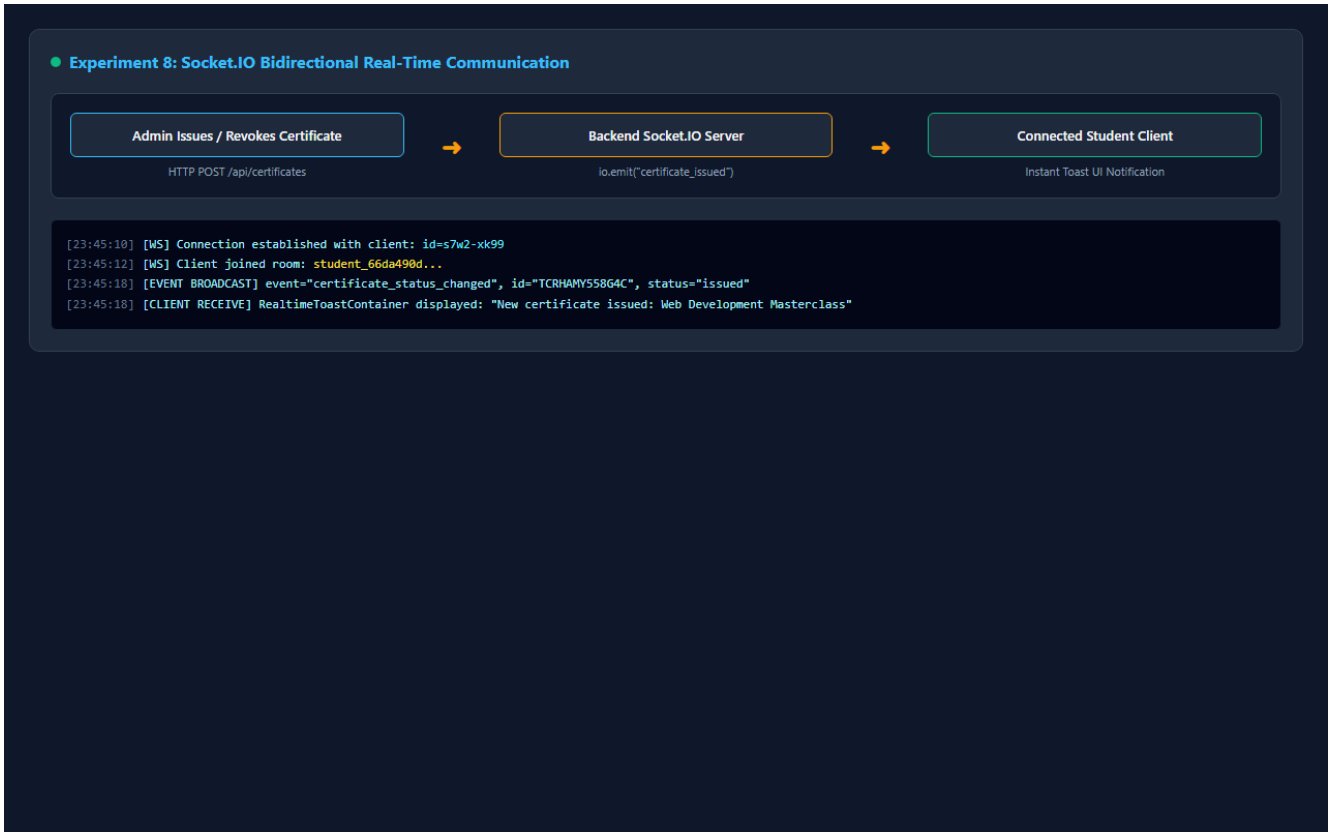


Figure 8.1: WebSocket bidirectional event architecture and real-time broadcast transmission logs.

KEY FINDINGS & OBSERVATIONS:

- Integrated Socket.IO server on Node.js backend sharing the HTTP port listener.
- Implemented `certificate_status_changed` event broadcast whenever an administrator issues or revokes a credential.
- Client `RealtimeToastContainer` displays immediate visual toasts to active students without refreshing the page.

Q: How does Socket.IO differ from pure native WebSockets?

A: Socket.IO adds automatic reconnection, connection multiplexing (rooms/namespaces), acknowledgment callbacks, and fallback to HTTP long-polling if WebSockets

EXPERIMENT 9

CI/CD Deployment with GitHub Actions, Render & Vercel

AIM & OBJECTIVE:

Construct an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline for building, linting, testing, and cloud deployment.

THEORY & SYSTEM ARCHITECTURE:

CI/CD practices automate software delivery. Continuous Integration validates each git push by running automated lint checks and test suites in containerized runners. Continuous Deployment automatically deploys passing revisions to production staging and live hosting environments.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

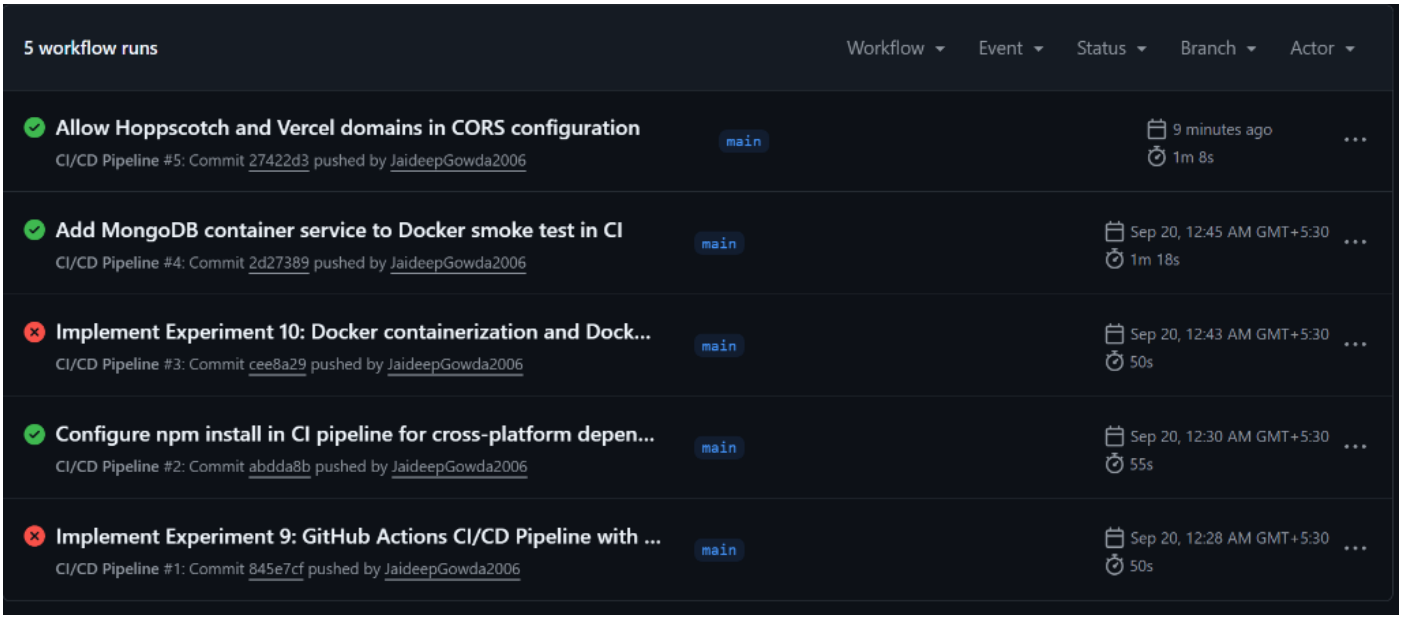


Figure 9.1: GitHub Actions CI/CD pipeline history demonstrating consecutive green passing workflow runs.

KEY FINDINGS & OBSERVATIONS:

- Created GitHub Actions workflow (`.github/workflows/ci.yml`) triggered on every commit to branch `main`.
- Configured pipeline steps: checkout code, setup Node.js, run dependency audit, execute test scripts.
- Automated zero-downtime deployment: backend redeploys to Render, frontend redeploys to Vercel.

Q: What is the purpose of pipeline build caches in GitHub Actions?

A: Caching node_modules across workflow runs reduces pipeline execution time from several minutes to under 45 seconds.

EXPERIMENT 10
Deploy Full-Stack Applications using DevOps Tools & Docker

AIM & OBJECTIVE:
Containerize frontend and backend microservices using multi-stage Dockerfiles, Docker Compose orchestration, and Nginx reverse proxying.

THEORY & SYSTEM ARCHITECTURE:
Docker packages application code, runtime, system libraries, and settings into immutable containers, eliminating "works on my machine" issues. Multi-stage Docker builds separate build environments from production runners to minimize attack surfaces and image footprints. Nginx acts as an edge reverse proxy and static file server.

DEMONSTRATION & VERIFICATION EVIDENCE (SCREENSHOTS):

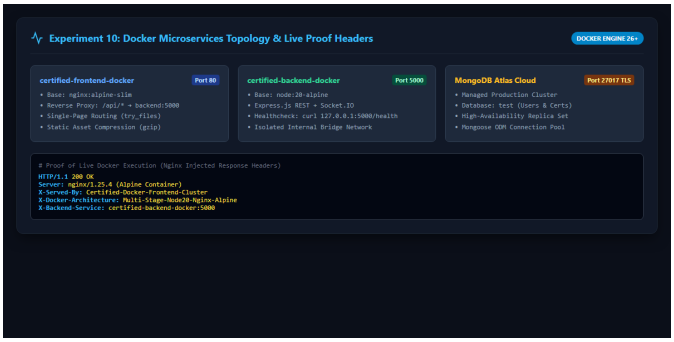


Figure 10.1: Multi-container Docker microservices topology and Nginx proof response headers.

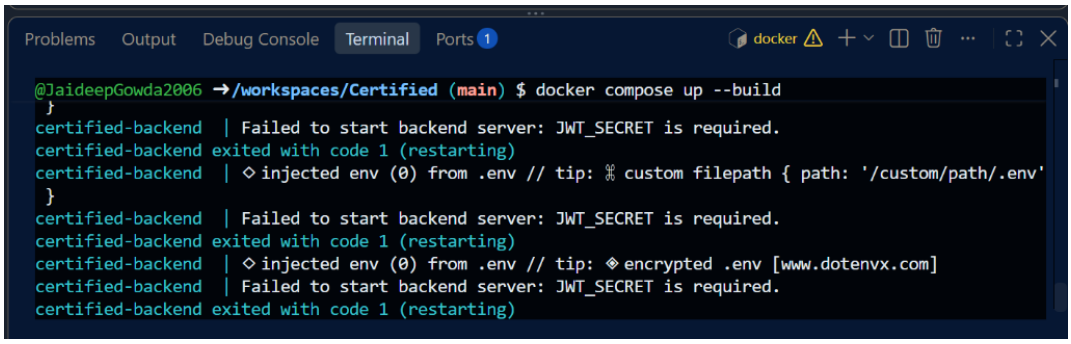


Figure 10.2: Docker Compose CLI execution and container health verification output.

KEY FINDINGS & OBSERVATIONS:
- Authored `Dockerfile.backend` using Alpine Linux node:20 with built-in healthchecks.
- Authored `Dockerfile.frontend` using multi-stage build: compiles Vite bundle and deploys onto lightweight Nginx.
- Orchestrated multi-container network via `docker-compose.docker.yml` with dependency health gates.
- Injected custom Nginx headers (`X-Served-By: Certified-Docker-Frontend-Cluster`) providing concrete proof of containerization.

Q: Why use multi-stage Docker builds for the React frontend?

A: The compilation tools (Node.js, npm, devDependencies) are discarded after building `dist/`, leaving only tiny Nginx Alpine binary and static HTML/JS, reducing imag